



南京大學

数据管理基础
实验手册（二）

实验手册三

前言

实验目的

实验数据说明

如何快速定位表结构或字段

实验部分

1. 查询每个登录活动赠送宝石总数

实验目的

实验数据

实验任务

拓展任务-改进login_bonus_detail

提示

2. 统计连续两个月都有登录奖励的短期活动

实验目的

实验数据

实验任务

3. 统计卡池时间

实验目的

实验数据

实验任务

扩展任务-EXPLAIN分析

实验目的

题目描述

四.提交方式

前言

本实验手册旨在通过实际操作，帮助大家掌握 **多表查询** 的核心技能。

实验目的

多表设计与关联查询

学习关系型数据库的表设计原则，包括主键、外键约束以及索引的使用。

学习如何使用 JOIN （内连接、外连接、自连接）进行多表关联查询，并理解不同连接方式的适用场景。

通过实际案例，分析复杂查询（如子查询、聚合函数、分组查询）的应用，提升 SQL 编写能力。

实验数据说明

- 本次实验所使用的 main.sql 数据文件来源于 《赛马娘（国服）》 游戏的解包数据，其中包含了该游戏中所有赛马娘角色及相关系统的完整数据结构与内容。
- 原始数据格式为 SQLite 数据库文件
- 使用 Navicat 工具将其 强制转换为 MySQL 8.2.0 版本兼容的 .sql 脚本文件
- 因此本次实验数据为一个 可直接导入 MySQL 的 SQL 文件
- 文件总长度约 16 万行，防止同学直接将 SQL 与题目整体丢进 GPT，实现“0 参与”

如何快速定位表结构或字段

建议使用 VSCode / Notepad++ / 任意文本编辑器 打开 main.sql ，并利用：

目标	搜索方式
查找建表语句	搜索： CREATE TABLE `表名`
查找字段位置	搜索： `字段名`
全局模糊搜索	Ctrl + F （全文件搜索）

实验部分

1. 查询每个登录活动赠送宝石总数

实验目的

1. 熟悉 多表 JOIN 操作。
2. 理解如何 聚合多个字段的数据。

实验数据

使用以下两张表

1. login_bonus_data: 登录活动基础信息

- `id` : 活动 ID
- `start_date` , `end_date` : 活动开始结束日期
- `count_num` 每一次登录活动会有多少次
- `type` 、 `disp_order` 等其他字段（本次实验用不到）

2. login_bonus_detail: 活动每一次的详细数据

- `login_bonus_id` : 关联活动 ID
- `count` : 第几次奖励
- `item_id ~ item_id_5` : 每次最多 5 种奖励物品的 ID
- `item_num ~ item_num_5` : 对应最多 5 种奖励物品的数目

实验任务

1. 查询每个登录活动的总钻石数量

- 将 `login_bonus_data` 与 `login_bonus_detail` 关联。
- 统计每个活动钻石数量的总和（把每一次的奖励的都加起来）。其中宝石的 `item_id = 43`。
- 输出列：
 - `login_bonus_id` : 活动 ID
 - `total_diamond` : 该活动总共赠送宝石数量

2. 排序

- 按 `total_diamond` 降序排列。

如何统计每一次活动有多少宝石奖励

```

1  SUM(
2      IF(item_id = 43, item_num, 0) +
3      IF(item_id_2 = 43, item_num_2, 0) +
4      IF(item_id_3 = 43, item_num_3, 0) +
5      IF(item_id_4 = 43, item_num_4, 0) +
6      IF(item_id_5 = 43, item_num_5, 0)
7  )

```

拓展任务-改进 login_bonus_detail

原数据库中，login_bonus_detail 表中每日奖励的物品信息最多只能记录 5 种奖励：

```

1  item_id, item_num
2  item_id_2, item_num_2
3  item_id_3, item_num_3
4  item_id_4, item_num_4
5  item_id_5, item_num_5

```

- 这种设计 **扩展性差**，如果未来活动需要赠送更多奖励，就必须修改表结构。

我们希望在扩展任务中能够完成下面的三个任务

1. 优化表结构，支持每次赠送任意数量和种类的奖励
2. 将现有表数据迁移到新表结构。
3. 能在新表结构上再次完成**钻石总数查询**任务。

注意：本次实验需要提交建表语句，插入语句和查询语句。以及提交 **钻石总数** 的查询结果即可（查询每一次登录活动的钻石总数）

提示

1. 可以创建一个新表例如 login_bonus_detail_new，而不是直接在 login_bonus_detail 上面修改
2. 可能需要使用 UNION ALL, 参考下面

```

1  SELECT d.id, o.item_category, o.item_id, o.item_num
2  FROM login_bonus_detail_old o
3  JOIN login_bonus_detail d
4      ON o.login_bonus_id = d.login_bonus_id AND o.count = d.count
5  WHERE o.item_id IS NOT NULL AND o.item_num > 0
6  UNION ALL
7  ... -- 类似的 SELECT 分别处理 item_id_2 ~ item_id_5

```

3. 使用 `INSERT INTO ... SELECT ...` 来实现迁移

2. 统计连续两个月都有登录奖励的短期活动

实验目的

1. 熟悉 自连接 (self JOIN) 技术。
2. 掌握 日期处理和月份计算。

实验数据

使用下面一张表

1. `login_bonus_data`: 登录活动基础信息

- `id` : 活动 ID
- `start_date` , `end_date` : 登录活动开始结束日期
- `type` 、 `disp_order` 等其他字段 (本次实验用不到)

实验任务

1. 筛选短期活动

- 条件: 登录活动持续时间 ≤ 31 天

2. 统计连续两个月都有活动的情况

- 方法: 自连接 `login_bonus_data` 表, 找到不同活动的两条记录, 满足:
 - i. 登录活动 1 与 登录活动 2 为不同活动
 - ii. 登录活动2的 `start_date` 在 登录活动1的下一个个月内 (例如活动1在 2023-01 开始, 活动2在 2023-02 开始)

1. 输出列:

- `activity_1_id` : 第一条活动 ID
- `activity_2_id` : 第二条活动 ID
- `month_1` : 第一条活动月份
- `month_2` : 第二条活动月份

提示1

可以使用 DATE_FORMAT 函数得到月数然后使用减法

```
1 DATE_FORMAT(start_date, '%Y-%m') AS month_1
```

提示2

- `a1.id <> a2.id` : 确保是不同活动
- `DATE_ADD(a1.start_date, INTERVAL 1 MONTH)` : 下一个月
- `DISTINCT` 去重重复匹配的行。

3. 统计卡池时间

注意：本题必须使用关键词 JOIN

实验目的

1. 熟悉大于两张表的多表连接
2. JOIN 中使用复杂条件，而不仅仅是等值匹配
3. 掌握GROUP BY操作

实验数据

使用下面三张表

1. **gacha_available**: 卡池可抽取物品表
 - `gacha_id` : 卡池 ID, 标识所属卡池
 - `card_id` : 物品 ID
 - `card_type` : 物品类型
 - `is_pickup` : 物品是否为 `pickup`
2. **gacha_data**: 卡池信息
 - `id` : 卡池 ID (与 `gacha_available.gacha_id` 对应)
 - `start_date` / `end_date` : 卡池开始和结束时间
3. **text_data**: 文本信息
 - `category` : 文本类别, 不同物品类型对应不同 `category`
 - `index` : 唯一 ID (对应物品的 `card_id`)
 - `text` : 物品名称或描述文本

实验任务

1. 统计每个卡池里面标记为 `pickup` 的物品的文本信息，同时输出卡池的开始和结束时间

由于一个卡池有多个物品，所以需要把所有物品的文本信息用逗号隔开，示例输出如下：

gacha_id	start_date	end_date	texts
10001	2021-03-02	2038-01-19	[优雅曲线]目白麦昆, [喜乐无边]东海帝王, [无声无瑕]无声铃鹿, [星光跃动]小栗帽.....

2. 要求：

- 只考虑 `gacha_available.is_pickup = 1` 的物品。 `is_pickup=0` 的要过滤掉
- 聚合文本信息之后使用逗号隔开
- `gacha_available.card_id` 对应 `text_data.index`，同时需要保证 `gacha_available.card_type` 与 `text_data.category` 匹配。映射关系如下：

gacha_available.card_type	text_data.category
1	4
2	75

提示：在查询物品文本时，需要同时满足两个条件：

- 需要使用 三表 JOIN: `gacha_available → gacha_data → text_data`
- `gacha_available.card_id = td.index`
- 如果物品类型 `card_type=1`，对应 `text_data.category=4`，如果物品类型 `card_type=2`，对应 `text_data.category=75`，在 SQL 查询中，可以使用 **CASE 语句** 来实现 `card_type` 与 `category` 的映射关系。
- 拼接文本可以使用 `GROUP_CONCAT(text ' ', ')`
- 时间处理函数推荐 `DATE_FORMAT + FROM_UNIXTIME`

3. 输出列：

- `gacha_id`：卡池 ID
- `start_date`：卡池开始时间
- `end_date`：卡池结束时间

- `text` : 物品信息, 逗号隔开

扩展任务-EXPLAIN分析

参考 <https://segmentfault.com/a/1190000023565685>

实验目的

- 了解 MySQL `EXPLAIN` 输出的含义。
- 学会解释查询计划中各字段的作用

题目描述

在你的 统计卡池时间 查询语句 前加上 `EXPLAIN` , 查看执行计划。根据 `EXPLAIN` 输出, 回答下列问题:

1. 查询从哪张表开始扫描, 接着扫描哪些表
 - a. 主要考察 `id` `table` 列
2. 分辨查询的类型, 是普通查询还是联合查询还是子查询
 - a. 主要考察 `select_type` 列
3. 以何种方式去访问我们的数据

可能会涉及索引, 但是不影响我们了解type字段的含义

- a. 主要考察 `type` 列
4. filtered列以及row列的含义解释
 5. Extra 字段中有哪些提示信息?
 6. 改变 JOIN 顺序
 - 尝试调整 SQL 查询中表的书写顺序, 例如先 `JOIN text_data` , 再连接其他表 (如 `gacha_available` 、 `gacha_data`) 。
 - 保证查询逻辑不变的前提下, 使用 `EXPLAIN` 查看执行计划是否发生变化
 - 记录实验结果。
 7. 使用 `STRAIGHT_JOIN` 强制执行顺序
 - 将查询中的普通 `JOIN` 改为 `STRAIGHT_JOIN` , 例如:

```
1 SELECT ...
2 FROM gacha_available STRAIGHT_JOIN gacha_data STRAIGHT_JOIN text_data
```

- 强制 MySQL 按书写顺序访问表，不允许优化器自动调整。
 - 使用 `EXPLAIN` 查看执行计划变化
 - 记录实验结果
8. 思考分析解释一下，改变join的顺序以及使用STRAIGHT_JOIN 所产生的现象的原因

四.提交方式

提交一个压缩包，包含所有题目的查询结果以及要回答的问题

打包为 `[学号+姓名].zip` 提交至moodle。